每日晨间简报

> **如果说我看得更远,那是因为我站在巨人的肩膀上。**
> *If I have seen further it is by standing on the shoulders of giants.*
> 艾萨克·牛顿 · 1675 年致罗伯特·胡克的信

今日一首歌　Here Comes the Sun — The Beatles
1969 年的老歌,木吉他打底的轻快民谣摇滚,三分钟出头。适合早上刚起来、拉开窗帘的时候听。

今天做 CSES 1072 Two Knights,CodeCrafters 接着做单引号这一关(ni6),后面附上下一关 tg6。

§1 新闻

科技与 AI

1. **Sakana AI 提出让模型自我改进的 MASS 方法**
   Sakana AI 10月12日的文章称,一个 27B 开放权重模型经两轮自我训练,每个输出 token 的得分升到 1.2–1.6 倍。(AIHOT)
2. **测试:智能体“组队”花费更多,质量提升很小**
   The Decoder 10月11日报道,Vals AI 测得团队的成本是单个智能体的 1.8–5.1 倍,四组对比里只有一组明显更好。(AIHOT)
3. **DeepSeek 据报融资至少 120 亿美元**
   Recode China AI 10月11日汇总,彭博社称腾讯参投;CNBC 援引消息人士称,规模可能增至 150 亿美元。 (AIHOT)
4. **SpaceXAI 账号改名为 SpaceXSI**
   IT之家 10月11日报道,马斯克发截图称官方账号已改名;此前特朗普下令政府部门改用“超级智能”一词。 (AIHOT)
5. **亚马逊:谈数据中心不再签保密协议**
   TechCrunch 10月11日报道,亚马逊称与地方政府谈数据中心时不再用保密协议,微软今年早些时候已这样做。(AIHOT)

编程与开发

1. **Let's Encrypt 证书有效期将缩到 64 天**
   Linuxiac 10月10日报道,2027年2月10日起签发的证书有效期从 90 天改为 64 天,2028 年计划再降到 45 天。
2. **SQLite 3.54 发布**
   Linuxiac 10月10日报道,项目方的基准测试显示比上一版快约 4%;新版不再支持 Windows XP。
3. **OpenClaw v2026.10.1 发布**
   OpenClaw 10月11日在 X 上称,新版合并了 402 位贡献者的 2,425 个 PR,新增“发现已安装技能”等功能。
4. **有帖子称谷歌面试试点“读代码”环节**
   Yuchen Jin 10月11日在 X 上转述,候选人 60 分钟内读 200–500 行陌生代码、找 bug,全程可用 Gemini。
5. **Yandex 称一处数据中心遭袭后停运**
   半岛电视台 10月11日报道,Yandex 称,它在俄罗斯弗拉基米尔州的一处数据中心在无人机袭击后停运。

财经与市场

1. **本周美股财报季开始,大银行周二打头**
   FXEmpire 10月11日列出的日程,周二摩根大通、高盛、花旗、富国银行公布财报,周三是美国银行和摩根士丹利。
2. **周三公布美国 9 月 CPI**
   TradingKey 10月11日称,市场预期 9 月 CPI 同比约 3.7%,8 月为 3.4%;核心 CPI 预期约 2.5%。
3. **台积电周四公布三季度完整业绩**
   TradingKey 10月11日称,台积电三季度营收约 1.49 万亿新台币,同比增长约 50%,高于市场预期。
4. **美联储 9 月加息后,市场倾向 10 月暂停**
   TradingKey 10月11日称,目前利率区间为 3.75%–4.00%,下一次议息会议在 10月27–28日。
5. **马斯克:星链在印度要“让人买得起”**
   半岛电视台 10月11日报道,对手 Jio 平台正筹备上市,目标估值 1060 亿美元;星链没有公布在印度的价格。

国内外要闻

1. **利雅得机场遇袭,沙特称至少 12 人死亡**
   半岛电视台 10月11日报道,沙特指袭击为胡塞武装所为,约 300 人受伤;特朗普称美国可能加入对胡塞的作战。
2. **吉布提外海移民船倾覆**
   半岛电视台 10月11日报道,国际移民组织称至少 139 人死亡、84 人失踪,船上原有 300 多人。
3. **巴基斯坦在西北一省实行紧急状态**
   半岛电视台 10月11日报道,开伯尔-普赫图赫瓦省由伊姆兰·汗的政党执政;司法部长称先由省督接管六个月。
4. **深圳新皇岗口岸今天开通**
   新华社 10月12日报道,这个连接深圳和香港的口岸重建后,通关时间预计从约 30 分钟缩到约 5 分钟。
5. **内蒙古出动两千多人防堵境外草原火**
   新华社 10月12日报道,火从蒙古国烧向边境,截至 11 日晚 9 时没有进入中国境内,扑救仍在进行。

KKKKhazix / AIHOT

一套可以自己搭的“行业热点站”:自动收集资讯、用大模型挑选和改写、每天出一份日报。这份简报的 AI 新闻就来自它的线上站点。

语言 **TypeScript**　星标 **约 0.7 万**　创建于 **2026年9月28日**　许可证 **MIT**　状态 **线上代码的快照,没有版本号**

▸ 这个项目有什么用

- **把“每天刷一堆网站”变成一个页面**。它从 RSS、网页列表、JSON 接口、X 账号、微信公众号等六种来源收集内容,汇总到一个站点里。
- **用大模型做编辑**。先粗筛,再按同一份标准独立打两次分,过线的才入选;入选的内容会被写成中文标题和摘要,外文全文也会翻译。
- **把同一件事的多篇报道归到一起**。几家媒体报同一件事时合并成一个“事件”,再按有多少个独立来源在报来排热度。
- **谁会用**。想给自己的行业(作者举了法律、人力资源、金融)搭一个资讯站的人:换掉信源名单和挑选标准就行,这些都集中放在 `industry/` 目录里。

▸ 怎么用

需要先装好 Docker,并准备一个 OpenAI 兼容的模型 API Key(README 举的例子有 DeepSeek、千问、智谱)。然后:

```
git clone https://github.com/KKKKhazix/AIHOT.git myhot
cd myhot
node scripts/init-env.ts --llm-key <你的模型 API Key>
docker compose up -d --build
```

把 `<你的模型 API Key>` 整个换成自己的 key;`myhot` 是文件夹名,可以随便起。启动后浏览器打开 `http://localhost:3000`,后台在 `/admin`,管理员密码在 `.env` 文件的 `ADMIN_PASSWORD` 里。README 说一两分钟后开始有内容,第一批资料大约半小时处理完。

▸ 为什么这么多人关注

- **涨得快**。仓库 9月28日 创建,两周左右到了约 7,200 个星标,在近三周新建的仓库里排第十。
- **切中“信息太多、看不过来”**。AI 消息每天成百上千条,很多人想要的是“有人替我挑好、写成中文”;这个项目把整条流水线都公开了。
- **挑选标准是公开、可改的**。提示词和入选门槛都放在仓库里,还带一个叫 SelectBench 的工具,用自己标注的样本检验挑得准不准。
- **为 AI 智能体留了出口**。除了网页,还提供 RSS、公开 API、MCP 和 `llms.txt`,别的程序和智能体可以直接读它的结果,这正是眼下的热点。

▸ 要注意的

> 作者自己写明:他是设计师出身,不是专业开发者,代码是和 AI 一起重写的,可能有写得不好的地方;不保证很快回复 Issue;仓库只是线上代码的快照,不保证每次更新都同步;仓库里不含线上站点的真实信源名单,只带 18 个公开的海外 AI 资讯源做示范;AIHOT 的名称和 Logo 不在许可范围内,自己搭站要换名字。运行它要持续调用模型 API,会产生费用。以上来自项目 README 和 GitHub 统计,没有实际安装验证。

▸ 和你的关系

它用的技术栈不是 Rust,但整个流程值得看:收集 → 打分 → 改写 → 归并 → 排序,每一步都是“把一个大任务拆成小步骤”的例子。它调用的也是 OpenAI 兼容的接口,和你 chat-cli 项目里调用 DeepSeek 用的是同一类接口。README 是中文的,读起来没有障碍。

§2 今日任务

标“必做”的是 CSES、CodeCrafters 和填写记录;其余是选做,按顺序往下做,做到哪算哪。

从这里开始 **CSES 1072 第 1 步:**在纸上画 2×2 和 3×3 的棋盘,弄清“两个马互不攻击的摆法”数的是什么。

☐ 01 CSES 第 1072 题 Two Knights

必做

▸ **Problem · 题目(英文转述)**

Your task is to count, for every **k = 1, 2, …, n**, the number of ways to place **two knights** on a **k × k chessboard** so that they do not **attack each other**.

> 你的任务:对 k = 1, 2, ..., n 中的每一个 k,数出在 k × k 的棋盘上放两个马、并且它们互不攻击,一共有多少种摆法。

Input. The only input line has one integer n.

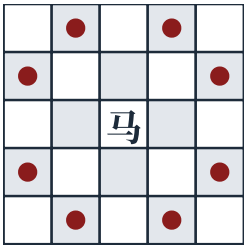
> **输入。**只有一行,一个整数 n。

Output. Print n integers, one per line: the answers for k = 1, 2, …, n.

> **输出。**输出 n 个整数,每行一个:依次是 k = 1, 2, ..., n 的答案。

Constraints. $1 \le n \le 10000$.

> **数据范围。**n 最大是 1 万。k = 10000 时棋盘有 1 亿个格子,答案大约是 5×10^{15},远超 32 位整数,要用 64 位;k 有 1 万个,每个 k 必须用式子直接算,不能真的去枚举摆法。



马走“日”字:中间的马能攻击到 8 个圆点

样例输入	样例输出
8	0 6 28 96 252 550 1056 1848

▸ **英文关键词**

`knight` (国际象棋的)马
`chessboard` 棋盘
`attack each other` 互相攻击
`the number of ways` 方法数、摆法数
`place … on …` 把…放在…上
`for each k` 对每一个 k

▸ **拆解步骤**

第 1 步 **读懂题,弄清“数的是什么”。**
在纸上画 2×2 和 3×3 的棋盘,弄清“两个马互不攻击的摆法”数的是什么:两个马一模一样,不分先后,只看占了哪两个格子。
☑ 确认:2×2 的棋盘上任意两格都互不攻击,一共 6 种,和样例第 2 行对上;1×1 放不下两个马,所以是 0。

第 2 步 **先不管攻击:一共有多少种摆法?**
棋盘有 k×k 个格子,从里面选两个不同的格子,有多少种选法?把它写成 k 的式子。
☑ 确认:k=2 时算出 6,k=3 时算出 36,k=4 时算出 120。

第 3 步 **弄清“互相攻击”的两个马长什么样。**
在纸上画一个 2×3 的小长方形(2 行 3 列),找出里面所有“互相攻击”的两格。再想想 3×2(3 行 2 列)的小长方形是不是一样。
☑ 确认:每个 2×3 或 3×2 的小长方形里,正好有 2 对互相攻击的位置(两条“对角”)。

第 4 步 **数一数 k×k 的棋盘里能放下多少个这样的小长方形。**

先数 2×3 的:它的左上角可以放在哪些行、哪些列?再数 3×2 的。把“互相攻击的对数”写成 k 的式子。

☑ 确认:k=3 时互相攻击的有 8 对(36 − 28),k=4 时有 24 对(120 − 96)。

第 5 步 **把两个式子合起来,先手算。**

答案 = 第 2 步的总数 − 第 4 步互相攻击的对数。

☑ 确认:手算 k=5 得 252,k=8 得 1848,和样例对上。

第 6 步 **写成代码并提交。**

把第 5 步填进下面骨架的 count 函数,k 从 1 循环到 n。

☑ 确认:输入 8,输出和样例的 8 行完全一样;再提交到 CSES。

▸ **提示**

① 全程用 `i64`:k×k 最大 1 亿,再乘一次就超过了 32 位整数的范围。② k=1 和 k=2 不需要单独处理——检查一下你的式子在这两个值上是不是自然得到 0 和 6。

▸ **Rust 代码骨架**

```rust
use std::io::Read;

// k×k 的棋盘上,两个马互不攻击的摆法数
fn count(k: i64) -> i64 {
    // 在这里写:
    // 1) 不管攻不攻击,一共多少种摆法
    // 2) 互相攻击的摆法有多少种
    // 3) 返回 1) 减 2)
    k
}

fn main() {
    let mut s = String::new();
    std::io::stdin().read_to_string(&mut s).unwrap();
    let n: i64 = s.trim().parse().unwrap();

    let mut out = String::new();
    for k in 1..=n {
        out.push_str(&format!("{}\n", count(k)));
    }
    print!("{out}");
}
```

☐ 02 CodeCrafters shell · ni6 Single quotes(单引号)

必做

这一关的内容和步骤编号与上一份简报相同。你在记录里写了想把下一关一起看,下一张卡就是 tg6(双引号)。

▸ Stage · 关卡要求(英文转述)

In this stage, your shell learns to understand **single quotes**.

> 这一关要让你的 shell 看懂**单引号**。

Every character between two single quotes is taken **literally**. Special characters such as `$`, `*`, `~` and the backslash lose their special meaning.

> 两个单引号之间的每个字符都**按字面**处理。`$`、`*`、`~` 和反斜杠这类特殊字符不再有特殊含义。

Whitespace inside single quotes is **preserved**: spaces are not collapsed and do not split the text into separate arguments.

> 单引号里的**空白**原样**保留**:多个空格不会合并,也不会把文字切成几个参数。

Quoted strings that are **adjacent** (no space between them) are joined into one **argument**. Empty quotes `''` add nothing.

> **紧挨着**的引号字符串(中间没有空格)会拼成同一个**参数**。空引号 `''` 什么也不加。

The tester starts your shell and runs commands such as `echo 'shell hello'`, `echo 'world test'` and `cat '/tmp/file name' '/tmp/file name with spaces'`. `cat` is a normal system program; you only need to pass it the right file names.

> 测试程序会启动你的 shell,运行 `echo 'shell hello'`、`echo 'world test'`、`cat '/tmp/file name' '/tmp/file name with spaces'` 这类命令。`cat` 是系统自带的程序,你只需要把正确的文件名传给它。

输入的命令	期望输出
`echo 'hello  world'`	`hello  world`
`echo hello  world`	`hello world`
`echo 'hello''world'`	`helloworld`
`echo hello''world`	`helloworld`

▸ 英文关键词

- `single quotes` 单引号
- `literally` 按字面(不做特殊解释)
- `whitespace` 空白(空格、制表符)
- `preserved / collapsed` 保留 / 合并压缩
- `adjacent` 相邻、紧挨着
- `argument` 参数

▸ 拆解步骤

第 1 步 **找到现在“把一行切成参数”的那段代码。**

多半是 `split_whitespace()` 之类。先弄清它在哪里被调用:echo、type、cd、运行外部程序是不是都用它。

☑ 确认:输入 echo hello world,现在输出 hello world。

第 2 步 **把切分单独抽成一个函数,行为先不变。**

写 `parse_args(line) -> Vec<String>`:一个字符一个字符扫描,遇到空白就结束当前的词。让原来的地方都改用它。

☑ 确认:echo hello world、pwd、cd、type echo 都和之前一样。

第 3 步 **加一个状态:现在是否在单引号里。**

遇到单引号就切换这个状态,引号本身不放进参数;在引号里时,空白当作普通字符。

☑ 确认:echo 'hello world' 输出 hello world(空格都在)。

第 4 步 **处理“紧挨着就拼接”。**

引号结束时不要结束当前的词;只有引号外面的空白才结束一个词。

☑ 确认:echo 'hello''world' 和 echo hello''world 都输出 helloworld。

第 5 步 **检查外部程序也拿到正确的参数。**

自己建两个带空格名字的文件,用 cat 加单引号读它们。

☑ 确认:cat '/tmp/file name' 能打印文件内容,而不是报“找不到 /tmp/file”。

第 6 步 **跑测试并提交。**

运行 `codecrafters test`,通过后 `git commit` 再 `codecrafters submit`(或 git push)。

☑ 确认:这一关显示通过。

▸ **提示**

> ① 只需要两个变量:当前正在拼的词(String)和“是否在单引号里”(bool)。② 循环结束后,最后一个词还没放进结果,别漏掉。

▸ **Rust 代码骨架**

```rust
fn parse_args(line: &str) -> Vec<String> {
    let mut args: Vec<String> = Vec::new();
    let mut cur = String::new();
    let mut in_single = false;

    for ch in line.chars() {
        match ch {
            '\'' => {
                // 切换 in_single;引号本身不放进 cur
            }
            c if c.is_whitespace() && !in_single => {
                // cur 不为空:放进 args,然后清空 cur
            }
            c => cur.push(c),
        }
    }
    // 循环结束:处理最后一个词
    args
}
```

☐ 03 CodeCrafters shell · 下一关 tg6 Double quotes(双引号)

必做

这是 ni6 之后紧接着的一关,按你在记录里说的一起发来。它是在 ni6 写好的 `parse_args` 上接着改,所以先做完 ni6 再做它;今天只想先看一看也可以。

▸ Stage · 关卡要求(英文转述)

In this stage, your shell learns to understand **double quotes**.

> 这一关要让你的 shell 看懂**双引号**。

Inside double quotes, spaces and tabs are **preserved**. Characters that normally split arguments are treated **literally**. (Special handling of `$` and the backslash comes in later stages.)

> 双引号里面,空格和制表符原样**保留**;平时用来切分参数的字符在这里**按字面**处理。(`$` 和反斜杠的特殊处理留到后面的关卡。)

Quoted strings next to each other, or next to unquoted text, are **concatenated** into one argument.

> 紧挨着的引号字符串,或者引号字符串紧挨着没加引号的文字,会**拼接**成同一个参数。

Quoted strings **separated** by whitespace remain separate arguments.

> 用空白**隔开**的引号字符串,仍然是各自独立的参数。

A single quote **enclosed** in double quotes is just an ordinary character.

> **包在**双引号里的单引号,只是一个普通字符。

The tester sends `echo` commands with double-quoted arguments, and then runs `cat` on double-quoted file paths that contain spaces and quote characters.

> 测试程序会发送带双引号参数的 `echo` 命令,然后用 `cat` 读取路径里带空格和引号字符、用双引号括起来的文件。

输入的命令	期望输出
`echo "hello"world`	`helloworld`
`echo "hello" "world"`	`hello world`
`echo "shell's test"`	`shell's test`

▸ 英文关键词

`double quotes` 双引号
`concatenate` 拼接
`separated by` 被…隔开
`enclosed in` 被…包住
`delimiter` 分隔符
`later stages` 后面的关卡

▸ 拆解步骤

第 1 步 **先确认起点。**
ni6 做完后,`parse_args` 里有一个“是否在单引号里”的状态。读一遍自己的代码,说出:遇到单引号、遇到空白、遇到其他字符,各做什么。
☑ 确认:echo 'a b' 输出 a b;echo "a b" 现在输出的是带引号的 "a b"(还不认识双引号)。

第 2 步 **再加一个状态:是否在双引号里。**
遇到双引号就切换这个状态,引号本身不放进参数。
☑ 确认:echo "hello" 输出 hello,没有引号。

第 3 步 **让双引号里的空白不切词。**
现在“空白结束一个词”的条件要改:在单引号里或在双引号里,空白都只是普通字符。
☑ 确认:echo "hello   world" 输出 hello   world;echo "hello" "world" 输出 hello world。

第 4 步 **处理两种引号互相套着的情况。**
在双引号里遇到单引号,它是普通字符,不切换单引号状态;反过来,在单引号里遇到双引号也一样。
☑ 确认:echo "shell's test" 输出 shell's test;echo 'say "hi"' 输出 say "hi"。

第 5 步 **检查拼接和外部程序。**

试紧挨着的写法,再建一个名字里带空格的文件,用 cat 加双引号读它。

☑ 确认:echo "hello"world 输出 helloworld;cat "/tmp/file name" 能打印文件内容。

第 6 步 **跑测试并提交。**

运行 `codecrafters test`,通过后提交。

☑ 确认:这一关显示通过,前面的 ni6 也仍然通过。

▸ **提示**

> ① 两个 bool 不会同时为真:在单引号里时不会进入双引号状态,反过来也一样。② 改完后把 ni6 的四条命令再手动跑一遍,确认没有改坏。

▸ **Rust 代码骨架**

```rust
fn parse_args(line: &str) -> Vec<String> {
    let mut args: Vec<String> = Vec::new();
    let mut cur = String::new();
    let mut in_single = false;
    let mut in_double = false;

    for ch in line.chars() {
        match ch {
            // match 的分支可以带条件(叫"守卫"):
            // 模式 if 条件 => { ... }
            '\'' if !in_double => {
                // 和 ni6 一样:切换 in_single
            }
            // 在这里加一个分支:遇到双引号怎么办?
            // 它的条件应该是什么?
            c if c.is_whitespace() => {
                // 这个条件现在不够了:
                // 在哪两种状态下,空白不能切词?
            }
            c => cur.push(c),
        }
    }
    // 循环结束:处理最后一个词
    args
}
```

☐ *04* Rust 知识点 · 第 2 讲:变量、可变性与整数类型

选做

一句话:变量是给一个值起的名字。Rust 里的名字默认不能改值,要改就得明说;每个数还有固定的类型和大小,放不下就是“溢出”。这一讲解决两个问题:什么时候写 `mut`,以及整数该选哪种类型。

书里讲的

- **变量默认不可变。** `let x = 5;` 之后不能再给 x 赋别的值。像用钢笔在表格里填了一个数:想改,得一开始就说明这一格是用铅笔写的,也就是 `let mut x = 5;`。
- **遮蔽(shadowing)。**可以用 `let` 再声明一个同名变量,新的把旧的“盖住”。它和 `mut` 不同:遮蔽是造了一个新变量,所以连类型都可以换。常见用法是把读进来的文字转成数字,还沿用原来的名字。
- **常量。** `const LIMIT: i64 = 1_000_000;` 永远不变,必须写类型,名字用大写。适合放“题目里给的上限”这类固定的数。
- **整数有很多种。** `i` 开头的有正负,`u` 开头的只有 0 和正数;后面的数字是占多少位:`i32` 能放到大约 21 亿 (2×10^9),`i64` 能放到大约 9×10^{18}。`usize` 专门用来做下标和长度。不写类型时,整数默认是 `i32`。
- **溢出。**算出来的数超过了类型能放的范围就叫溢出。像只有 4 位的里程表,9999 再加 1 就出问题了。用 `cargo run`(调试模式)时,溢出会让程序直接崩溃并报错;用 `--release` 编译时不检查,数会悄悄“绕回去”变成错的值。

例 1(能编译):不可变、可变、遮蔽和常量。输出 `5 15 42 1000000`。

```rust
fn main() {
    let x = 5; // 不可变:之后不能再改
    let mut sum = 0; // 可变:加了 mut 才能改
    sum += x;
    sum += 10;

    // 遮蔽:用同一个名字重新声明,类型也可以换
    let line = "42";
    let line: i64 = line.parse().unwrap();

    const LIMIT: i64 = 1_000_000;
    println!("{x} {sum} {line} {LIMIT}");
}
```

例 2(这段编不过):给不可变的变量再次赋值。编译器报 `E0384: cannot assign twice to immutable variable `total``,意思是“total 不可变,不能赋第二次值”;下面的 `help:` 会直接建议改成 `let mut total = 0;`。

```rust
fn main() {
    let total = 0;
    for i in 1..=3 {
        total += i; // total 没有声明成 mut
    }
    println!("{total}");
}
```

书里没细讲,但你应该知道的

- **为什么默认不可变。**Python、JavaScript 里任何变量随时能改,读代码时得一直记着“它现在是多少”。Rust 反过来:没写 `mut` 的变量,看到声明就知道它的值到最后都不变。读别人的代码(和一周后读自己的代码)时,只需要盯住带 `mut` 的那几个。编译器还会提醒你“这个 mut 没用上”,可以顺手删掉。

- **刷题时整数怎么选。**一条够用的习惯:题目里的数一律用 `i64`,下标和长度用 `usize`。判断会不会溢出只要估一下最大值:两个 10^9 相乘是 10^{18},`i64` 刚好放得下,`i32` 放不下。Python 的整数没有上限,所以从 Python 转过来的人最容易在这里出错。
- **不同类型不能直接混着算。**`i32` 和 `i64` 相加会报 `mismatched types`。用 `as` 转换:`x as i64`。从小类型转到大类型是安全的;从大转到小会直接截断,不报错,所以要先确认数值放得下。
- **小心无符号数相减。**`u64` 的 3 − 5 不是 −2,而是溢出。需要“差”或者可能出现负数时,用 `i64`;只想要两数的距离,用 `abs_diff`。昨天那道螺旋题里“离对角线几格”可能是负数,就是这个道理。
- **标准库里配套的方法。**`checked_add / checked_mul` 在溢出时返回 `None`,适合“不确定会不会溢出”的地方;`pow(2)` 求平方;`max / min / abs` 直接点在数字后面用。数字里可以加下划线分组,`1_000_000` 比 `1000000` 好读。

例 3(能编译):用 `i64` 放大数,用 `checked_add` 检查溢出,用 `usize` 做下标。输出依次是 `4999999950000000`、`None`、`4000000000`、`30 3`。

```
fn main() {
    let k: i64 = 10_000;
    let sq = k * k; // 1 亿,i64 放得下
    let big = sq * (sq - 1) / 2; // 约 5×10^15,仍然放得下
    println!("{big}");

    // 不确定会不会溢出时,用 checked_ 系列,溢出返回 None
    let a: i32 = 2_000_000_000;
    println!("{:?}", a.checked_add(a)); // None
    println!("{}", a as i64 + a as i64); // 4000000000

    // usize 用来做下标和长度
    let v = vec![10, 20, 30];
    let i: usize = 2;
    println!("{} {}", v[i], v.len());
}
```

例 4(能编译):无符号数相减的三种写法。输出依次是 `2`、`-2`、`None`。

```
fn main() {
    let a: u64 = 3;
    let b: u64 = 5;
    println!("{}", a.abs_diff(b)); // 2
    println!("{}", a as i64 - b as i64); // -2
    println!("{:?}", a.checked_sub(b)); // None
}
```

▸ **常见错误**

- 忘了写 `mut`,在循环里累加时报 E0384 → 在声明处加 `mut`。反过来,不需要改的变量不要习惯性地加 `mut`。
- 中间结果溢出:最终答案放得下,但算到一半的乘积放不下 → 参与运算的变量从一开始就声明成 `i64`,而不是算完再转。

▸ **小练习**

不运行,判断下面这段会发生什么,怎么改?

```
fn main() {
    let n: i32 = 100_000;
    let pairs = n * (n - 1) / 2;
    println!("{pairs}");
}
```

> **和今天任务的联系:**CSES 1072 的第 2 步要算“k×k 个格子里选两个”,k 最大 1 万,k×k 是 1 亿,再乘一次约 10^{16},所以骨架里 `count` 的参数和返回值都是 `i64`;上面的小练习就是同一种式子换了个数。CodeCrafters 的 `parse_args` 里,`cur`、`args`、`in_single` 都要在循环里被修改,所以都声明成 `let mut`;而 `line` 只读不改,不需要 `mut`。

练习答案:10 万 × 99999 约等于 10^{10},超过了 i32 的上限(约 2×10^9)。这里的数都是写死的,编译器在编译时就能算出来,所以直接拒绝编译,报 this arithmetic operation will overflow;如果 n 是运行时读进来的,就会变成运行时崩溃(调试模式)或得到错的数(release 模式)。改法:把 n 声明成 i64。 · 对应书里第 3 章:3.1 变量和可变性、3.2 数据类型,想深入时可以去读。

☐ *05* 英语 + AI 精读 选做

▸ 先看 8 个关键词

`multi-agent system` 多智能体系统
`subagent` 子智能体(帮手)
`workflow` 工作流程(谁做什么、按什么顺序)
`token` 词元(模型读写文字的小单位)
`judge` 评判
`distillation` 蒸馏(把能力传给一个模型)
`recursive self-improvement` 递归式自我改进
`benchmark` 基准测试

▸ One Model, Many Workers · 一个模型,许多"工人"

Most people use an AI model as a single assistant: you ask, and it answers. A newer idea is the **multi-agent system**. Here, one main agent splits a big task into small jobs and hands them to helpers called **subagents**. Each subagent works on its own piece, and the main agent puts the results together.

> 大多数人把 AI 模型当作一个助手来用:你问,它答。一个更新的想法是**多智能体系统**:一个主智能体把大任务拆成小活,交给叫作**子智能体**的帮手。每个子智能体负责自己那一块,主智能体再把结果合起来。

The plan that says who does what, and in what order, is called a **workflow**. A good workflow matters a lot. With a poor one, the helpers repeat each other's work and burn many **tokens**, the small units of text that models read and write. More tokens mean more time and more money.

> 规定"谁做什么、按什么顺序做"的那份安排,叫作**工作流程**。流程好不好很要紧:流程差的时候,帮手们会重复彼此的工作,白白烧掉很多 **token**——模型读写文字时用的小单位。token 越多,花的时间和钱就越多。

On October 12, the research company Sakana AI described a method called MASS, short for Multi-Agent Self-Supervision. One shared model plays every role. It runs a team of virtual subagents to solve tasks. It also searches for better workflows, and it **judges** its own outputs to pick the best ones.

> 10月12日,研究公司 Sakana AI 介绍了一种叫 MASS 的方法,全称是"多智能体自监督"。所有角色都由同一个模型来扮演:它运行一队虚拟的子智能体去解决任务,同时寻找更好的工作流程,还**评判**自己的输出,从中挑出最好的。

Then comes the key step. The work of the best team is used as training data for the same model. Passing skill from a larger system into one model like this is often called **distillation**. The improved model can then run the cycle again. Doing this again and again is known as **recursive self-improvement**.

> 接下来是关键的一步:最好的那支团队的工作过程,被拿来当作训练同一个模型的数据。像这样把一个更大系统的本事传进一个模型里,通常叫作**蒸馏**。变强了的模型可以把这个循环再跑一遍。这样一遍一遍地做,就是所谓的**递归式自我改进**。

Sakana AI ran two cycles on a 27B open-weights model. It reports that, after the second cycle, the score per output token was 1.2 to 1.6 times the base model's level on four research **benchmarks**. These are the company's own results, on its own tasks. Still, the idea is worth knowing: a model may get better not only from human data, but also from carefully chosen examples of its own teamwork.

> Sakana AI 在一个 27B 的开放权重模型上跑了两轮。它报告说,第二轮之后,在四个研究类**基准测试**上,每个输出 token 的得分达到原始模型的 1.2 到 1.6 倍。这些是这家公司在自己设计的任务上得出的结果。不过这个思路值得了解:模型变强,靠的不一定只是人写的数据,也可以是精心挑出来的、它自己"团队合作"的例子。

相关新闻:Recursive Self-Improvement through Multi-Agent Self-Supervision(Sakana AI,10月12日)。做法:先读英文,再看翻译核对;读完用中文写 3 句话总结。

☐ *06* 英语原著阅读 · Atomic Habits 育儿附录 · 第 1 / 131 段:开头 + THE 1ST LAW

选做

今天读	Atomic Habits 附录《How to Apply These Ideas to Parenting》,文件 `Parenting_Appendix.pdf`,第 1–3 页(读到 THE 2ND LAW 之前)。小节:开头 + THE 1ST LAW:Make it obvious。
路线	全部材料共 131 段,今天是第 1 段。这篇附录一共 4 段,今天是它的第 1 段。全文骨架:开头回顾习惯回路(cue → craving → response → reward)和四条法则,然后一条法则一节,每节都是“法则是什么 → 用在孩子身上的做法 → 例子”。上一段:没有,这是起点。下一段:THE 2ND LAW:Make it attractive,讲父母做榜样、同伴的影响和给孩子选择权。

▸ 读之前 · 带着问题读

– 作者说把这些方法用在孩子身上,和用在自己身上相比,最大的不同是什么?

– “幼儿园式整理法”有哪几个特点?先猜,再到原文里找那张 5 条的清单。

▸ 结构地图

(1) 开头:回顾习惯回路的四步和对应的四条法则;正着用让行为更容易,反着用(inversion)让行为更难。

(2) 提醒:这里是在塑造别人的习惯,所以更难;给的只是起点,每个家庭要自己试。

(3) 第一法则对应 cue(提示):提示越明显,越容易被注意到并照着做。

(4) 做法一:学幼儿园教室——分区、一次只做一件事、东西放在用的地方、每样东西有固定位置、重要的东西看得见;可以用颜色标签。

(5) 做法二:习惯叠加(habit stacking),公式是“做完现有的习惯之后,做新的习惯”;新习惯要小,也可以把想做的事放在后面当奖励。

▸ 关键词

`underlie` 是…的基础

`neurological feedback loop` 神经反馈回路

`inversion` 反过来(反向运用)

`cue` 提示、信号

`signify` 表示、意味着

`be acted upon` 被照着去做

`habit stack` 习惯叠加

`incentivize` 用奖励鼓励

▸ 句型

The more …, the more likely …	越…就越可能…(本段的说法是 cues that are more obvious will be more likely to ...) The clearer the instructions are, the more likely people are to follow them. 说明写得越清楚,人们越可能照着做。
After [A], I will [B].	做完 A 之后,我就做 B(习惯叠加的公式) After I open my laptop, I will read one page of English. 打开电脑之后,我先读一页英文。

三遍读法:第一遍只看标题、每段第一句和加粗的词,对照结构地图;第二遍从头读到尾,不查词,不懂的画线;第三遍只回到画线的地方,对照关键词弄懂。

▸ 读完之后

(1) 四条法则分别对应习惯回路的哪一步?

(2) 幼儿园的例子里,颜色起什么作用?

(3) 作者说,真正的目标是让孩子写一小时作业时,一开始应该怎么做?

用英文写一句话总结这一段:_____

答案:① Cue—make it obvious;craving—make it attractive;response—make it easy;reward—make it satisfying。② 让“东西放哪、该做什么”一眼就看得出来,也就是把提示变明显。③ 先建立很小的叠加习惯,把环境准备好(例如进门后把作业拿出来放到桌上)。 · 参考总结:The first law is to make the cue obvious, for example with color labels and habit stacks.

☐ *07* chat-cli 项目 · 第 2 天:结构体与错误处理 选做

目标:把手写的 JSON 换成有类型的结构体,程序只打印回复文本,出错时给出看得懂的信息。

▸ **加依赖**

```
cargo add serde --features derive
cargo add anyhow
```

▸ **任务清单**

- ☐ 定义 `Message`,请求和响应共用
- ☐ 定义 `ChatRequest`,用它替换 `json!` 宏
- ☐ 定义 `ChatResponse`、`Choice`、`Usage`,用 `.json::<ChatResponse>()` 解析响应
- ☐ `main` 的返回类型改为 `anyhow::Result<()>`,把所有 `.unwrap()` 换成 `?`
- ☐ 发送后检查状态码,不成功时打印状态码和响应正文
- ☐ 打印回复文本,再打印一行 token 用量

结构体的起点,其余的照着写:

```
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize, Debug, Clone)]
struct Message {
    role: String,
    content: String,
}

#[derive(Serialize)]
struct Thinking {
    #[serde(rename = "type")]
    kind: String,
}
```

▸ **提示**

- `type` 是 Rust 关键字,不能当字段名,所以用 `#[serde(rename = "type")]`。
- serde 默认忽略结构体里没定义的字段,只写你用得到的就行。
- `Usage` 只需要 `prompt_tokens`、`completion_tokens`、`total_tokens` 三个字段。
- `finish_reason` 可能为空,类型用 `Option<String>`。
- 检查状态用 `resp.status().is_success()`,失败时用 `anyhow::bail!("...")` 返回错误。

▸ **常见错误码**

状态码	含义	怎么办
401	key 错误	检查环境变量
402	余额不足	充值
400 / 422	请求格式或参数不对	打印请求体,和文档对照
429	请求太频繁	等一会儿再试
500 / 503	服务端故障或过载	稍后重试

▸ **常见坑**

– 解析失败时不要猜,先用 `.text()` 把原始响应打出来,和结构体逐个字段对照。

完成标准 正常时只输出回复文本和用量;故意把 key 改错,程序不崩溃,而是打印出 401 和错误说明。

这一段照你的7 天指南写。做完后在记录页把 chat-cli 标为完成,简报才会进到第 3 天:多轮对话。

☐ *08* **读书:《1984》第一部 第 3–5 章** 选做

今天读:第一部 第 3–5 章(接着你读完的第 2 章)。

留意:第 3 章里主角对“过去”还记得多少;第 4 章他上班具体在做什么;第 5 章食堂里那位编词典的同事是怎么谈论语言的。

▸ **读完想一想**

– 主角的工作是怎样改动“过去”的?如果所有记录都能被改,一个人还能凭什么确认一件事发生过?

– 那位同事认为词越少越好。词变少了,人能想的事情会不会跟着变少?

☐ *09* **运动:跑步** 选做

今天:跑步。

☐ 热身:快走 5 分钟,再活动脚踝、膝盖和髋

☐ 慢跑 30 分钟,能完整说出一句话的强度;中途喘不过来就改成快走,缓过来再跑

☐ 拉伸:小腿、大腿前后侧和臀部各拉一拉

☐ *10* **填写今晚的记录** 必做

在记录页填今天的进度。

做了一点也写上“做到第几步、大约做了多久”;选做的做了就勾上,没做留空(读了今天的英语原著那一段,就勾“读了英文文章”,明天换下一段)。明天的简报从这里写的位置接着排。
